

Revisión Papers Semana 3

Marcelo Astete Chávez

Software Bill of Materials Adoption- A Mining Study from GitHub

1. Problema que se está resolviendo

El problema que se está resolviendo es la baja adopción de Listas de Materiales de Software (SBOM) en proyectos de código abierto, lo que aumenta el riesgo de utilizar componentes obsoletos y vulnerables. **Esto es crítico para la seguridad del software, especialmente en el contexto de nuevas regulaciones y estándares de la cadena de suministro de software.** La investigación busca entender cómo y con qué frecuencia los desarrolladores actualizan sus archivos SBOM para mejorar la seguridad y la gestión de dependencias.

2. Idea Propuesta

La nueva idea propuesta es un enfoque para mejorar la adopción y gestión de SBOM en proyectos de código abierto, facilitando su integración y actualización en el ciclo de vida del desarrollo de software. Este enfoque incluye la fusión de modelos de características y la utilización de estándares como SPDX y CycloneDX para estructurar y distribuir los archivos SBOM de manera más efectiva. Además, se sugiere que la adopción de SBOM puede ser impulsada por eventos políticos y normativos, como la orden ejecutiva del presidente de la república sobre ciberseguridad.

3. Puntos Positivos

- Mejora de la Seguridad: La adopción de SBOM ayuda a identificar y gestionar vulnerabilidades en componentes de software, aumentando la seguridad general del software.
- Cumplimiento Normativo: Facilita el cumplimiento de nuevas regulaciones y estándares de ciberseguridad, como los impulsados por el gobierno de EE. UU. y la Unión Europea.
- Colaboración y Transparencia: Promueve la colaboración entre desarrolladores y la transparencia en la gestión de dependencias, lo que puede mejorar la calidad del software y la confianza de los usuarios.

4. Puntos Negativos

- Baja Adopción: A pesar de su importancia, la adopción de SBOM en proyectos de código abierto sigue siendo baja, lo que limita su efectividad en la mejora de la seguridad.

- Complejidad de Implementación: Integrar SBOM en el ciclo de vida del desarrollo de software puede ser complejo y requerir cambios significativos en las prácticas actuales de los desarrolladores.

- Falta de Estándares Universales: No existe una métrica aceptada para evaluar cuantitativamente la adopción de SBOM, lo que dificulta la comparación y el seguimiento de su implementación en diferentes proyectos.

5. Trabajo Futuro

Investigación Empírica Adicional: Realizar estudios empíricos más profundos sobre la adopción de SBOM en proyectos de código abierto, incluyendo encuestas y análisis de casos específicos para entender mejor las barreras y facilitadores.

Desarrollo de Herramientas: Crear herramientas automatizadas que simplifiquen la generación y actualización de SBOM, facilitando su integración en el flujo de trabajo de los desarrolladores.

Evaluación de Impacto: Investigar el impacto de la adopción de SBOM en la seguridad del software y la gestión de vulnerabilidades, proporcionando métricas y estudios de caso que demuestren sus beneficios tangibles.

6. Calificación (1 a 5)

La calificación para este artículo es nota 5 debido a que el análisis de la adopción del SBOM es relevante y bien fundamentado.

7. Puntos de Discusión

Discutir las principales barreras que enfrentan los desarrolladores para adoptar SBOM, como la falta de herramientas adecuadas y la complejidad de implementación.

Examinar la evolución de los estándares como SPDX y CycloneDX, y cómo su adopción puede ser impulsada por la comunidad de desarrolladores y las organizaciones de estándares.

On the Accuracy of GitHub's Dependency Graph

1. Problema que se está resolviendo

El problema que se está resolviendo es la inexactitud en el gráfico de dependencias de GitHub, que puede llevar a errores en la gestión de dependencias en proyectos de código abierto. Esto incluye casos de repositorios de origen no encontrados y repositorios no disponibles, lo que afecta la precisión y la seguridad de las herramientas que dependen de esta información. La investigación busca identificar y analizar estas inexactitudes para mejorar la fiabilidad del gráfico de dependencias.

2. Idea Propuesta

La nueva idea propuesta es mejorar la precisión del gráfico de dependencias de GitHub mediante un análisis exhaustivo de las inexactitudes encontradas en proyectos de código abierto. Esto incluye proporcionar información cualitativa sobre las causas de estas inexactitudes y sus implicaciones para el uso de herramientas como Dependabot y generadores de SBOM. La investigación sugiere que esta mejora podría aumentar la efectividad y seguridad en la gestión de dependencias en el desarrollo de software.

3. Puntos Positivos

- **Análisis Integral:** Se complementa el análisis cuantitativo con uno cualitativo, lo que proporciona una comprensión más profunda de las inexactitudes en el gráfico de dependencias.
- **Muestras Significativas:** Se utilizan muestras aleatorias estadísticamente significativas para garantizar la validez de las conclusiones, aumentando la credibilidad del estudio.
- **Relevancia Práctica:** Los hallazgos tienen implicaciones directas para la seguridad y gestión de dependencias en proyectos de código abierto, beneficiando a desarrolladores y herramientas de software.

4. Puntos Negativos

- **Inexactitudes Persistentes:** A pesar del análisis, el gráfico de dependencias sigue sin reflejar constantemente el estado actualizado de las dependencias, lo que puede comprometer la seguridad.
- **Limitaciones en la Causalidad:** No se aborda la validez interna ni se investiga la causalidad, lo que limita la profundidad del análisis y la comprensión de las razones detrás de las inexactitudes.
- **Dependencia de Datos Externos:** La precisión del estudio depende de la disponibilidad de repositorios y archivos de manifiesto/bloqueo,

lo que puede afectar la recuperación de datos y los resultados.

5. Trabajo Futuro

La propuesta de trabajo futuro incluye realizar un estudio para evaluar en qué medida la imprecisión en el gráfico de dependencias de GitHub afecta a investigaciones previas que lo han utilizado. Además, se planea replicar el estudio para determinar si ha habido mejoras en la construcción del gráfico de dependencias a lo largo del tiempo. También se sugiere explorar otros lenguajes de programación para ampliar la comprensión de las inexactitudes en diferentes contextos.

6. Calificación (1 a 5)

La calificación para este artículo es nota 5 debido a que se realiza un análisis exhaustivo del tema y se encuentra respaldado por evidencia.

7. Puntos de Discusión

Cómo las inexactitudes en el gráfico de dependencias pueden afectar la toma de decisiones de los desarrolladores, especialmente en términos de seguridad y gestión de actualizaciones.

La importancia de la retroalimentación de los usuarios sobre el uso del gráfico de dependencias para identificar áreas de mejora y aumentar la precisión y utilidad de la herramienta.